Module III: Data Structures in Python

Topics: Strings and String Operations | Lists and List Methods | Tuples and Sets | Dictionaries and Dictionary Operations | List Comprehension | Regular Expressions (Regex)

3.1 Strings and String Operations

What is a string?

A string is a sequence of characters enclosed in single quotes, double quotes, or triple quotes. Strings are **immutable**: once created, they cannot be changed in place. Any operation that appears to modify a string actually creates a new one.

```
s1 = 'Hello'
s2 = "Python"
s3 = '''This is a
multi-line string'''
```

Indexing and slicing

Every character has a position. Indexing starts at 0 from the left and at -1 from the right.

```
 P   y   t   h   o   n
 0   1   2   3   4   5
-6  -5  -4  -3  -2  -1
```

```
word = "Python"
print(word[0])      # P
print(word[-1])     # n
print(word[1:4])    # yth
print(word[:3])     # Pyt
print(word[3:])     # hon
print(word[::2])    # Pto
print(word[::-1])   # nohtyP
```

Slice syntax: `string[start:stop:step]`. The stop index is never included.

Basic operations

Operation	Example	Result
Concatenation	`"Hi" + " there"`	`Hi there`
Repetition	`"ab" * 3`	`ababab`
Length	`len("Python")`	`6`
Membership	`"th" in "Python"`	`True`
Comparison	`"apple" < "banana"`	`True`

Immutability in action:

```
s = "Python"
s[0] = "J"        # TypeError: 'str' object does not support item assignment
s = "J" + s[1:]   # works: creates a new string "Jython"
```

Common string methods

Case conversion

```
s = "hello World"
print(s.upper())        # HELLO WORLD
print(s.lower())        # hello world
print(s.title())        # Hello World
print(s.capitalize())   # Hello world
print(s.swapcase())     # HELLO wORLD
```

Searching

```
text = "banana"
print(text.find("an"))      # 1  (returns -1 if not found)
print(text.index("an"))     # 1  (raises ValueError if not found)
print(text.count("a"))      # 3
print(text.startswith("ba"))  # True
print(text.endswith("na"))    # True
```

Cleaning and replacing

```
msg = "   welcome   "
print(msg.strip())          # 'welcome'
print(msg.lstrip())         # 'welcome   '
print(msg.rstrip())         # '   welcome'
print("a-b-c".replace("-", "+"))   # a+b+c
```

Splitting and joining

```
line = "red,green,blue"
colors = line.split(",")        # ['red', 'green', 'blue']
print("-".join(colors))         # red-green-blue
print("a b  c".split())         # ['a', 'b', 'c']
```

Checking content

```
print("abc123".isalnum())   # True
print("abc".isalpha())      # True
print("123".isdigit())      # True
print("   ".isspace())      # True
print("Hello".isupper())    # False
print("hello".islower())    # True
```

Alignment

```
print("Hi".center(10, "*"))   # ****Hi****
print("Hi".ljust(5, "."))     # Hi...
print("7".zfill(3))           # 007
```

String formatting

```
name = "Asha"
marks = 87.456

# 1. f-string (preferred)
print(f"{name} scored {marks:.2f}")

# 2. format() method
print("{} scored {:.1f}".format(name, marks))

# 3. % operator (older style)
print("%s scored %.2f" % (name, marks))
```

Escape sequences

Sequence	Meaning
`\n`	New line
`\t`	Tab
`\\`	Backslash
`\'` and `\"`	Quotes

A **raw string** (`r"C:\new\table"`) treats backslashes literally. This becomes important in regex.

Example program: palindrome check

```
def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]

print(is_palindrome("Nurses Run"))   # True
print(is_palindrome("Python"))       # False
```

3.2 Lists and List Methods

What is a list?

A list is an ordered, **mutable** collection that can hold items of different types. Lists are written in square brackets.

```
nums = [10, 20, 30, 40]
mixed = [1, "two", 3.0, True]
empty = []
nested = [[1, 2], [3, 4]]
from_range = list(range(5))     # [0, 1, 2, 3, 4]
```

Accessing and modifying

```
fruits = ["apple", "banana", "cherry", "date"]
print(fruits[0])        # apple
print(fruits[-1])       # date
print(fruits[1:3])      # ['banana', 'cherry']

fruits[1] = "blueberry"   # lists can be changed in place
print(fruits)             # ['apple', 'blueberry', 'cherry', 'date']
```

Nested access: `nested[1][0]` gives `3`.

Traversing a list

```
for f in fruits:
    print(f)

for i, f in enumerate(fruits):
    print(i, f)
```

List methods

Method	Purpose	Example
`append(x)`	Add x at the end	`a.append(5)`
`insert(i, x)`	Insert x at index i	`a.insert(1, 99)`
`extend(iterable)`	Add all items of another iterable	`a.extend([7, 8])`
`remove(x)`	Remove first occurrence of x	`a.remove(5)`
`pop(i)`	Remove and return item at i (last if omitted)	`a.pop()`
`clear()`	Remove all items	`a.clear()`
`index(x)`	Position of first x	`a.index(8)`
`count(x)`	Number of times x occurs	`a.count(2)`
`sort()`	Sort in place	`a.sort(reverse=True)`
`reverse()`	Reverse in place	`a.reverse()`

Method	Purpose	Example
`copy()`	Shallow copy	`b = a.copy()`

```
a = [3, 1, 4]
a.append(1)         # [3, 1, 4, 1]
a.insert(0, 9)      # [9, 3, 1, 4, 1]
a.extend([5, 6])    # [9, 3, 1, 4, 1, 5, 6]
a.remove(1)         # [9, 3, 4, 1, 5, 6]
last = a.pop()      # last = 6, a = [9, 3, 4, 1, 5]
a.sort()            # [1, 3, 4, 5, 9]
```

append vs extend

```
x = [1, 2]
x.append([3, 4])    # [1, 2, [3, 4]]  -> adds ONE item (a list)

y = [1, 2]
y.extend([3, 4])    # [1, 2, 3, 4]    -> adds each element
```

sort() vs sorted()

`sort()` changes the original list and returns `None`. `sorted()` returns a new list and leaves the original untouched.

```
marks = [72, 95, 60]
print(sorted(marks))      # [60, 72, 95]
print(marks)              # [72, 95, 60]

names = ["Ravi", "asha", "Meena"]
names.sort(key=str.lower)  # case-insensitive sort
```

Built-in functions on lists

```
nums = [4, 8, 15, 16, 23, 42]
print(len(nums), min(nums), max(nums), sum(nums))
print(sum(nums) / len(nums))     # average
```

Copying: a common trap

```
a = [1, 2, 3]
b = a            # b is another name for the SAME list
b.append(4)
print(a)         # [1, 2, 3, 4]  (a changed too!)

c = a.copy()     # or a[:] or list(a)
c.append(5)
print(a)         # [1, 2, 3, 4]  (unchanged)
```

For nested lists, `copy()` is shallow. Use `copy.deepcopy()` to copy inner lists as well.

Other useful operations

```
[1, 2] + [3]        # [1, 2, 3]
[0] * 4             # [0, 0, 0, 0]
3 in [1, 2, 3]      # True
del a[0]            # delete by index
```

Example program: second largest element

```
def second_largest(lst):
    unique = sorted(set(lst), reverse=True)
    return unique[1] if len(unique) > 1 else None

print(second_largest([10, 45, 20, 45, 30]))   # 30
```

3.3 Tuples and Sets

Tuples

A tuple is an ordered, **immutable** sequence written in parentheses. Use tuples for data that should not change, such as coordinates, dates, or RGB values.

```
t = (10, 20, 30)
single = (5,)         # the comma makes it a tuple; (5) is just an int
no_paren = 1, 2, 3    # also a tuple
empty = ()
```

```
print(t[0])        # 10
print(t[1:])       # (20, 30)
print(len(t))      # 3
t[0] = 99          # TypeError: tuples cannot be modified
```

Tuple methods: only two exist.

```
t = (1, 2, 2, 3)
print(t.count(2))    # 2
print(t.index(3))    # 3
```

Packing and unpacking

```
point = (4, 7)          # packing
x, y = point            # unpacking
print(x, y)             # 4 7

a, b = 1, 2
a, b = b, a             # swap values

first, *rest = (1, 2, 3, 4)
print(first, rest)      # 1 [2, 3, 4]
```

Why use tuples?

- Faster and lighter than lists
- Safe from accidental changes
- Can be used as dictionary keys (lists cannot)
- Functions can return multiple values as a tuple

```
def min_max(lst):
    return min(lst), max(lst)

low, high = min_max([5, 2, 9])
```

Sets

A set is an **unordered** collection of **unique** elements. It is mutable, but its elements must be immutable (numbers, strings, tuples).

```
s = {1, 2, 3, 3, 2}
print(s)                 # {1, 2, 3}  duplicates removed
empty_set = set()        # {} creates an empty dictionary, not a set
unique = set([1, 1, 2, 2, 3])
```

Sets have no indexing, because there is no order.

Set methods

Method	Purpose
`add(x)`	Add one element
`update(iterable)`	Add many elements
`remove(x)`	Remove x (error if absent)
`discard(x)`	Remove x (no error if absent)
`pop()`	Remove an arbitrary element
`clear()`	Empty the set

Mathematical set operations

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A | B)    # union:                {1, 2, 3, 4, 5, 6}
print(A & B)    # intersection:        {3, 4}
print(A - B)    # difference:          {1, 2}
print(A ^ B)    # symmetric difference:{1, 2, 5, 6}

print({1, 2} <= A)     # subset: True
print(A.isdisjoint({9}))   # True
```

Equivalent methods: `union()`, `intersection()`, `difference()`, `symmetric_difference()`, `issubset()`, `issuperset()`.

Common use: removing duplicates and fast membership testing.

```
names = ["a", "b", "a", "c", "b"]
print(list(set(names)))     # duplicates removed (order not guaranteed)
```

Comparison of the four sequence/collection types

Feature	List	Tuple	Set	Dictionary
Syntax	`[ ]`	`( )`	`{ }`	`{k: v}`
Ordered	Yes	Yes	No	Yes (insertion order)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Indexed by	Position	Position	None	Key

3.4 Dictionaries and Dictionary Operations

What is a dictionary?

A dictionary stores data as **key: value** pairs. Keys must be unique and immutable (strings, numbers, tuples). Values can be anything. Since Python 3.7, dictionaries remember insertion order.

```
student = {
    "name": "Kiran",
    "age": 19,
    "course": "BCA"
}
empty = {}
d = dict(a=1, b=2)
```

Accessing, adding, updating, deleting

```
print(student["name"])            # Kiran
print(student.get("grade"))       # None (no error)
print(student.get("grade", "NA")) # NA  (default value)

student["age"] = 20               # update
student["city"] = "Pune"          # add new key

del student["city"]               # delete a key
student.pop("age")                # delete and return value
```

Using `student["grade"]` for a missing key raises `KeyError`, while `get()` does not.

Dictionary methods

Method	Purpose
`keys()`	All keys
`values()`	All values
`items()`	All (key, value) pairs
`get(k, default)`	Safe lookup
`update(other)`	Merge another dictionary
`pop(k)`	Remove key and return value
`popitem()`	Remove last inserted pair
`setdefault(k, v)`	Return value, inserting k with v if missing
`clear()`	Remove everything
`copy()`	Shallow copy
`fromkeys(keys, v)`	New dict from keys with one value

```
d = {"a": 1, "b": 2}
print(d.keys())      # dict_keys(['a', 'b'])
print(d.values())    # dict_values([1, 2])
print(d.items())     # dict_items([('a', 1), ('b', 2)])

d.update({"b": 20, "c": 3})
print(d)             # {'a': 1, 'b': 20, 'c': 3}
```

Looping through a dictionary

```
for key in d:
    print(key, d[key])

for k, v in d.items():
    print(f"{k} -> {v}")
```

Membership

```
print("a" in d)         # True  (checks keys only)
print(20 in d.values()) # True
```

Nested dictionaries

```
class_data = {
    101: {"name": "Asha", "marks": 88},
    102: {"name": "Ravi", "marks": 76}
}
print(class_data[101]["name"])    # Asha
```

Dictionary comprehension

```
squares = {n: n**2 for n in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Example program: word frequency counter

```
text = "to be or not to be"
freq = {}
for word in text.split():
    freq[word] = freq.get(word, 0) + 1
print(freq)    # {'to': 2, 'be': 2, 'or': 1, 'not': 1}
```

Example program: sorting a dictionary by value

```
marks = {"Asha": 88, "Ravi": 76, "Meena": 92}
ranked = sorted(marks.items(), key=lambda item: item[1], reverse=True)
print(ranked)    # [('Meena', 92), ('Asha', 88), ('Ravi', 76)]
```

3.5 List Comprehension

Idea

A list comprehension is a short way to build a new list from an existing iterable in a single line.

General form:

```
[expression  for item in iterable  if condition]
```

From loop to comprehension

```
# Using a loop
squares = []
for n in range(1, 6):
    squares.append(n ** 2)

# Using a comprehension
squares = [n ** 2 for n in range(1, 6)]     # [1, 4, 9, 16, 25]
```

With a condition (filtering)

```
evens = [n for n in range(1, 21) if n % 2 == 0]
long_words = [w for w in ["sun", "python", "code"] if len(w) > 3]
```

With if-else (transforming)

When `else` is used, the condition moves **before** the `for`.

```
labels = ["even" if n % 2 == 0 else "odd" for n in range(5)]
# ['even', 'odd', 'even', 'odd', 'even']
```

More examples

```
words = ["hello", "world"]
upper = [w.upper() for w in words]           # ['HELLO', 'WORLD']
lengths = [len(w) for w in words]            # [5, 5]

digits = [int(ch) for ch in "2024" if ch.isdigit()]   # [2, 0, 2, 4]

flat = [x for row in [[1, 2], [3, 4]] for x in row]   # [1, 2, 3, 4]

pairs = [(x, y) for x in range(2) for y in range(2)]
# [(0, 0), (0, 1), (1, 0), (1, 1)]
```

Set and generator forms

```
unique_lengths = {len(w) for w in ["a", "bb", "cc"]}   # {1, 2}
total = sum(n * n for n in range(10))                  # generator expression
```

When not to use it

If the logic needs several statements or becomes hard to read, use a normal loop. Readability matters more than saving a line.

3.6 Regular Expressions (Regex)

What is a regular expression?

A regular expression (regex) is a pattern used to search, match, and manipulate text. Typical uses are validating emails and phone numbers, extracting dates from text, and cleaning data. Python provides regex support through the built-in `re` module.

```
import re
```

Basic functions of the re module

Function	What it does
`re.match(p, s)`	Matches only at the **start** of the string
`re.search(p, s)`	Finds the **first** match anywhere
`re.fullmatch(p, s)`	Whole string must match
`re.findall(p, s)`	Returns a list of all matches
`re.finditer(p, s)`	Returns an iterator of match objects
`re.sub(p, repl, s)`	Replaces matches
`re.split(p, s)`	Splits string at matches
`re.compile(p)`	Pre-compiles a pattern for reuse

```
text = "My number is 98765 and code is 4321"

print(re.search(r"\d+", text).group())    # 98765
print(re.findall(r"\d+", text))           # ['98765', '4321']
print(re.match(r"My", text))              # match object
print(re.match(r"number", text))          # None
print(re.sub(r"\d+", "#", text))          # My number is # and code is #
print(re.split(r"\s+", "a  b   c"))       # ['a', 'b', 'c']
```

Always write patterns as **raw strings** (`r"..."`) so backslashes are not treated as escapes by Python.

Special characters (metacharacters)

Symbol	Meaning
`.`	Any single character except newline
`^`	Start of string
`$`	End of string
`*`	0 or more repetitions
`+`	1 or more repetitions

Symbol	Meaning
`?`	0 or 1 repetition
`{n}`	Exactly n repetitions
`{n,m}`	Between n and m repetitions
`[ ]`	Set of characters, e.g. `[aeiou]`
`[^ ]`	Any character NOT in the set
`|`	OR
`( )`	Grouping and capturing
`\`	Escape a special character

Character classes

Class	Meaning	Equivalent
`\d`	Digit	`[0-9]`
`\D`	Non-digit	`[^0-9]`
`\w`	Letter, digit or underscore	`[a-zA-Z0-9_]`
`\W`	Non-word character	`[^a-zA-Z0-9_]`
`\s`	Whitespace	space, tab, newline
`\S`	Non-whitespace	
`\b`	Word boundary	

Examples that build up

```
re.findall(r"[aeiou]", "education")        # ['e', 'u', 'a', 'i', 'o']
re.findall(r"\b\w{5}\b", "hello world python")   # ['hello', 'world']
re.findall(r"colou?r", "color colour")     # ['color', 'colour']
re.findall(r"a.c", "abc a-c aXc")          # ['abc', 'a-c', 'aXc']
re.search(r"^Py", "Python")                # match
re.search(r"on$", "Python")                # match
```

Groups

Parentheses capture parts of a match.

```
m = re.search(r"(\d{2})-(\d{2})-(\d{4})", "Date: 15-08-2024")
print(m.group())     # 15-08-2024
print(m.group(1))    # 15
print(m.group(2))    # 08
print(m.group(3))    # 2024
print(m.groups())    # ('15', '08', '2024')
print(m.start(), m.end())
```

Named groups make the code easier to read:

```
m = re.search(r"(?P<day>\d{2})-(?P<month>\d{2})", "15-08")
print(m.group("month"))   # 08
```

Greedy and non-greedy matching

By default, `*` and `+` match as much as possible. Adding `?` makes them match as little as possible.

```
html = "<b>bold</b><i>italic</i>"
print(re.findall(r"<.*>", html))     # ['<b>bold</b><i>italic</i>']   greedy
print(re.findall(r"<.*?>", html))    # ['<b>', '</b>', '<i>', '</i>']  non-
greedy
```

Flags

```
re.findall(r"python", "Python PYTHON python", re.IGNORECASE)
# ['Python', 'PYTHON', 'python']
```

Common flags: `re.IGNORECASE` (`re.I`), `re.MULTILINE` (`re.M`), `re.DOTALL` (`re.S`).

Practical validation programs

Mobile number (10 digits, starting with 6 to 9)

```
def valid_mobile(num):
    return re.fullmatch(r"[6-9]\d{9}", num) is not None

print(valid_mobile("9876543210"))   # True
print(valid_mobile("1234567890"))   # False
```

Email address (basic)

```
pattern = r"^[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}$"
print(bool(re.match(pattern, "student@college.edu")))   # True
print(bool(re.match(pattern, "student@college")))       # False
```

Simple password rule: at least 8 characters, one uppercase letter, one digit.

```
def strong(pw):
    return (len(pw) >= 8 and
            re.search(r"[A-Z]", pw) and
            re.search(r"\d", pw)) is not None
```

Extracting all email IDs from text

```
text = "Contact a@x.com or b.c@y.org for details"
print(re.findall(r"[\w.]+@[\w.]+\.\w+", text))
# ['a@x.com', 'b.c@y.org']
```

Using compile()

If the same pattern is used many times, compile it once.

```
phone = re.compile(r"\d{3}-\d{4}")
print(phone.findall("555-1234 and 777-9999"))
```

Quick Revision Summary

- **String:** immutable sequence of characters; supports indexing, slicing, and many built-in methods.
- **List:** ordered, mutable; use `append`, `insert`, `extend`, `remove`, `pop`, `sort`.
- **Tuple:** ordered, immutable; supports packing and unpacking.
- **Set:** unordered, unique elements; supports union, intersection, difference.

- **Dictionary:** key-value pairs; use `get`, `items`, `update`, `pop`.
- **List comprehension:** `[expr for x in iterable if cond]`.
- **Regex:** pattern matching with the `re` module; use raw strings, `search`, `findall`, `sub`, `split`.

Practice Questions

Short answer

1. Why are strings called immutable? Give an example.
2. Differentiate between `append()` and `extend()`.
3. What is the difference between `remove()`, `pop()` and `del` for a list?
4. Why can a tuple be used as a dictionary key but a list cannot?
5. How do you create an empty set? Why does `{}` not work?
6. Differentiate between `re.match()` and `re.search()`.
7. What is the difference between greedy and non-greedy matching?

Output prediction

```
# Q1
s = "programming"
print(s[2:8:2], s[::-3])

# Q2
a = [1, 2, 3]
b = a
b.append(4)
print(a)

# Q3
print({1, 2, 3} & {2, 3, 4}, {1, 2, 3} - {2, 3, 4})

# Q4
d = {"x": 1, "y": 2}
print(d.get("z", 0), len(d))

# Q5
print([x * x for x in range(6) if x % 2])
```

Programming exercises

1. Write a program to count vowels, consonants, digits and spaces in a string.
2. Accept a sentence and print the longest word in it.
3. Remove duplicates from a list while keeping the original order.
4. Given two lists, print elements common to both using sets.
5. Create a dictionary of student names and marks, then print the topper and the class average.
6. Count the frequency of each character in a string using a dictionary.
7. Use a list comprehension to generate all numbers from 1 to 100 divisible by both 3 and 5.
8. Write a regex-based program to validate a PAN card number (format: 5 letters, 4 digits, 1 letter).
9. Extract all dates in the format `dd/mm/yyyy` from a paragraph of text.
10. Replace all multiple spaces in a string with a single space using `re.sub()`.

Answers to output prediction

1. `orm` and `gmrr`. Here `s[2:8:2]` picks indices 2, 4, 6 and `s[::-3]` picks indices 10, 7, 4, 1.
2. `[1, 2, 3, 4]`
3. `{2, 3}` and `{1}`
4. `0 2`
5. `[1, 9, 25]`